

Proyecto 2: Implementación de un R-Tree

Castillo Prado, Leonardo Humberto

Mattos Gutierrez, Angel Daniel

Dasso Arana, Alvaro José

Algoritmos y Estructuras de Datos

3 de julio de 2026

1. Introducción

En este proyecto se implementa un R-Tree desde cero en C++ para almacenar y consultar objetos espaciales. La estructura permite organizar puntos geográficos mediante rectángulos mínimos envolventes, conocidos como MBRs (*Minimum Bounding Rectangles*), con el objetivo de acelerar consultas espaciales como búsqueda por rango y búsqueda de vecinos más cercanos o KNN frente a una solución ingenua como la búsqueda lineal que necesita recorrer todo el dataset, por lo que su costo crece directamente con la cantidad de objetos. En cambio, el R-Tree permite descartar subárboles completos cuando sus MBRs no son relevantes para la consulta.

2. Objetivos

El objetivo principal es implementar un R-Tree utilizando datos espaciales y compararlo contra una solución ingenua basada en búsqueda lineal.

La implementación considera las siguientes operaciones principales:

- Inserción de objetos espaciales.
- Actualización de MBRs después de modificaciones.
- División de nodos mediante split cuadrático.
- Consultas por rango rectangular.
- Consulta de vecinos más cercanos (KNN) usando un MinHeap.
- Comparación contra búsqueda lineal.
- Carga y visualización de datos espaciales reales.

3. Modelo de datos

La estructura trabaja con objetos espaciales representados por coordenadas geográficas. En la implementación, la longitud se usa como coordenada x y la latitud como coordenada y .

Las estructuras principales son:

- **SpatialObject**: Struct que almacena el id, nombre, categoría y coordenadas de un objeto.
- **MBR**: representa un rectángulo mínimo envolvente mediante `minX`, `minY`, `maxX` y `maxY`.
- **Entry**: representa una entrada dentro de un nodo. En hojas guarda un objeto real; en nodos internos guarda un MBR y un puntero a un hijo.

- **RTNode**: representa un nodo del R-Tree. Contiene entradas, un MBR, un indicador de hoja y un puntero al padre.
- **RTree**: representa la estructura completa, incluyendo la raíz, la capacidad máxima por nodo y las operaciones principales.

El MBR es la pieza central del R-Tree. Sus funciones más importantes son:

- **area()**: calcula el área del rectángulo.
- **expand()**: retorna el MBR mínimo que cubre dos rectángulos.
- **enlarge()**: calcula cuánto crecería un MBR al incluir otro.
- **intersects()**: verifica si dos MBRs se cruzan.
- **distanceTo()**: calcula la distancia mínima entre un punto y un MBR.

Consideramos todas estas operaciones con costo $O(1)$ porque solo operan sobre cuatro coordenadas.

4. Construcción del R-Tree

La inserción comienza convirtiendo el punto a insertar en un MBR degenerado, es decir, un rectángulo de área cero donde:

$$\min X = \max X = x, \quad \min Y = \max Y = y$$

Luego se ejecuta **chooseLeaf**, que baja desde la raíz hasta una hoja. En cada nodo interno se elige el hijo cuyo MBR tendría el menor crecimiento si recibiera el nuevo objeto. Si hay empate, se elige el MBR de menor área. Esta decisión es una heurística: no garantiza una organización óptima global, pero busca mantener los MBRs compactos.

Después de insertar el objeto en la hoja, se actualizan los MBRs hacia arriba mediante **adjustTree**. Esto es necesario porque el MBR de una hoja puede crecer, y ese cambio debe reflejarse en sus padres hasta llegar a la raíz.

Si un nodo supera su capacidad máxima M , se aplica **splitNode**. El proyecto usa split cuadrático. Este algoritmo elige dos entradas iniciales, llamadas semillas, que producirían el mayor desperdicio (*waste*) de área si quedaran juntas:

$$waste = area(MBR_1 \cup MBR_2) - area(MBR_1) - area(MBR_2)$$

Luego, las entradas restantes se reparten entre los dos grupos. Para cada entrada se calcula cuánto crecería el MBR del grupo A y cuánto crecería el MBR del grupo B. La entrada se asigna al grupo donde cause menor crecimiento. Si hay empate, se usan criterios secundarios como menor área actual o menor cantidad de entradas.

El split cuadrático intenta reducir la superposición entre MBRs, lo cual mejora las consultas posteriores.

5. Consultas implementadas

5.1. Consulta por rango

La consulta por rango recibe un rectángulo de búsqueda y retorna los objetos ubicados dentro de él. El algoritmo empieza en la raíz y revisa las entradas del nodo actual. Si el MBR de una entrada no intersecta el rectángulo de consulta, esa entrada se descarta junto con todo su subárbol.

Si la entrada sí intersecta, se continúa explorando. Cuando se llega a una hoja, se verifica si el punto del objeto está dentro del rectángulo consultado. Si lo está, se agrega al resultado.

La ventaja frente a búsqueda lineal es que el R-Tree puede podar regiones completas del espacio sin revisar objeto por objeto.

5.2. Consulta KNN

La consulta KNN busca los k objetos más cercanos a un punto. Para ello se utiliza un MinHeap, que permite procesar primero el candidato con menor distancia.

Los candidatos pueden ser nodos, MBRs u objetos reales. Cuando el candidato es un nodo, se insertan sus hijos en el heap usando la distancia mínima entre el punto de consulta y cada MBR. Cuando el candidato es un objeto real, se agrega al resultado. El algoritmo termina cuando se obtienen los k vecinos más cercanos.

La distancia punto-MBR funciona como una cota inferior. Si un MBR está como mínimo a distancia d del punto de consulta, ningún objeto dentro de ese MBR puede estar a una distancia menor que d . Por eso el algoritmo puede explorar primero las regiones más prometedoras sin perder exactitud.

6. Complejidad algorítmica

Sea n la cantidad de objetos almacenados, M la capacidad máxima de entradas por nodo, h la altura del árbol, k la cantidad de resultados y v la cantidad de candidatos visitados durante KNN.

Operación	Complejidad esperada	Peor caso
Operaciones sobre MBR	$O(1)$	$O(1)$
chooseLeaf	$O(h \cdot M)$	$O(n)$
Inserción sin split	$O(h \cdot M)$	$O(n)$
Split cuadrático	$O(M^2)$	$O(M^2)$
Inserción con splits	$O(h \cdot M^2)$	$O(n)$
Consulta por rango	$O(\log n + k)$	$O(n)$
KNN con MinHeap	$O(v \log v)$	$O(n \log n)$
Búsqueda lineal	$O(n)$	$O(n)$

Cuadro 1: Complejidades principales de la implementación

El costo de **chooseLeaf** es $O(h \cdot M)$ porque en cada nivel del árbol se revisan hasta M entradas para elegir el mejor hijo. Si el árbol está balanceado y M se considera constante, este costo se aproxima a $O(\log n)$.

El split cuadrático cuesta $O(M^2)$ porque compara pares de entradas para elegir las semillas iniciales. Además, durante la distribución de entradas restantes puede volver a evaluar candidatos para decidir qué entrada asignar a cada grupo.

La consulta por rango tiene costo esperado $O(\log n + k)$ cuando el árbol permite podar muchas ramas. Sin embargo, en el peor caso puede llegar a $O(n)$ si los MBRs se solapan demasiado o si la región consultada cubre gran parte del espacio.

En KNN, v representa la cantidad de candidatos visitados. Estos candidatos pueden ser nodos internos, nodos hoja, MBRs u objetos reales. Como el algoritmo usa un MinHeap, cada inserción o extracción cuesta $O(\log v)$. Por eso la consulta cuesta $O(v \log v)$. En el peor caso, si se visita casi todo el árbol, v puede acercarse a n , dando $O(n \log n)$.

La búsqueda lineal se usa como comparación porque revisa todos los objetos. Para consultas por rango, verifica uno por uno si cada punto cae dentro del rectángulo. Para KNN, calcula distancias contra todos los objetos y selecciona los más cercanos.

7. Datasets y visualización

El proyecto utiliza datos reales cargados desde la carpeta `assets`. El archivo `cities15000.txt` proviene de GeoNames y contiene ciudades con identificador, nombre, latitud, longitud y código de país. Estos datos se convierten a objetos `SpatialObject`.

También se utiliza `grido.json`, un archivo en formato JSON obtenido desde OpenStreetMap mediante Overpass API. JSON es un formato de texto para representar datos estructurados, y Overpass API permite consultar elementos geográficos de OpenStreetMap. En este caso, los puntos cargados se modelan como objetos espaciales de categoría `heladeria`.

La visualización se implementa con SDL. La interfaz permite observar los puntos cargados, hacer consultas por rango arrastrando con el mouse y ejecutar KNN con click derecho. También permite hacer zoom y desplazarse por el mapa. Esta parte ayuda a comprobar visualmente que los resultados del R-Tree coinciden con las consultas espaciales realizadas.

8. Comparación y medición

La comparación principal se realiza contra búsqueda lineal. Esta comparación es útil porque la búsqueda lineal representa la solución ingenua: simple, correcta, pero costosa para datasets grandes.

En la práctica, algunas consultas KNN con valores pequeños de k , como $k = 5$, pueden registrarse como 0 ms si se mide en milisegundos. Esto no significa que el algoritmo no trabaje, sino que la consulta tarda menos de un milisegundo y el valor se redondea. Para una medición más precisa quizás lo mejor sea medir en microsegundos o ejecutar muchas consultas y reportar el promedio.

La métrica más importante no es solo el tiempo, sino también la cantidad de candidatos o nodos visitados. El R-Tree es ventajoso cuando logra visitar muchos menos elementos que la búsqueda lineal.

Para la inserción usamos una heurística de menor crecimiento para elegir la hoja adecuada. Cuando un nodo se llena, el split cuadrático divide las entradas intentando reducir desperdicio de área y superposición. Aunque este split no garantiza una distribución óptima global, es una estrategia práctica y adecuada para mantener el árbol balanceado.